

HIGHLY AVAILABLE 3-TIER WEB APPLICATION ON AWS

Project guide • Architecture • Implementation • Validation • LinkedIn portfolio

1. Project overview

Build a web application on AWS that can serve traffic through a load balancer, scale EC2 capacity, keep the database in private subnets, store static objects in S3, and provide DNS, access control, and monitoring.

Learning objectives:

- Design a VPC with public and private subnets across at least two Availability Zones.
- Deploy EC2 instances in an Auto Scaling group and register them with an Application Load Balancer (ALB).
- Use Amazon RDS for MySQL with Multi-AZ deployment for database high availability.
- Use IAM roles and security groups to apply least-privilege access.
- Use S3 for static assets and CloudWatch for logs, metrics, alarms, and troubleshooting.
- Use Route 53 to map a domain name to the ALB.

2. Architecture at a glance

Request flow: User → Route 53 → ALB in public subnets → EC2 Auto Scaling group in application subnets → RDS MySQL in database subnets.

Supporting services: S3 stores static assets; IAM governs permissions; CloudWatch collects metrics/logs and triggers alarms.

Recommended production-oriented layout: one VPC across two Availability Zones; public subnets for the ALB; private application subnets for EC2; isolated database subnets for RDS. Use a NAT Gateway or VPC endpoints only where needed for outbound access. For a cost-conscious lab, understand that NAT Gateways, ALB, and RDS can incur charges.

3. AWS services and purpose

Service	Role in this project
Amazon VPC	Private network, CIDR, subnets, route tables, internet/NAT routing.
Amazon EC2	Runs the web/application server.
Application Load Balancer	Distributes HTTP/HTTPS requests and health-checks targets.
EC2 Auto Scaling	Maintains desired instance count and replaces unhealthy instances.
Amazon RDS for MySQL	Managed relational database; Multi-AZ supports failover.
Amazon S3	Object storage for static files, uploads, or artifacts.
AWS IAM	Roles, policies, and least-privilege permissions.
Amazon CloudWatch	Metrics, logs, dashboards, and alarms.
Amazon Route 53	DNS hosted zone and alias record to the ALB.

4. Step-by-step implementation

Step 1 — Plan the network

Choose a Region and a VPC CIDR such as 10.0.0.0/16. Select two Availability Zones. Plan public subnets for the ALB and private subnets for application and database resources. Avoid overlapping CIDRs.

Step 2 — Create the VPC and subnets

Create the VPC, enable DNS resolution and DNS hostnames, and create public, private-application, and private-database subnets in both AZs. Attach an Internet Gateway. Associate public route tables with a 0.0.0.0/0 route to the IGW. Keep application/database subnets private. If private instances need internet egress, configure NAT or use appropriate VPC endpoints.

Step 3 — Create security groups

ALB security group: allow inbound 80/443 from intended clients. EC2 security group: allow the application port (for example 80 or 8080) only from the ALB security group. RDS security group: allow TCP 3306 only from the EC2/application security group. Do not expose MySQL publicly.

Step 4 — Create the RDS MySQL database

Create a DB subnet group using private database subnets in at least two AZs. Create an RDS MySQL database, enable Multi-AZ for a high-availability deployment, set storage and backups, and keep Public access disabled. Store credentials securely (for example, AWS Secrets Manager) rather than hard-coding them. Confirm EC2-to-RDS connectivity.

Step 5 — Create IAM permissions

Create an EC2 instance role with only the permissions the application needs, such as reading a specific S3 bucket path and publishing logs. Attach the role through an instance profile. Avoid long-lived access keys on instances and avoid broad AdministratorAccess policies.

Step 6 — Prepare the application

Create a small web application or use a sample app. Configure it to read database settings from secure configuration, connect to RDS, and serve a health endpoint such as /health. Keep uploaded/static objects in S3 when appropriate. Do not put database passwords in source code or user data logs.

Step 7 — Create the launch template and Auto Scaling group

Create a launch template with the chosen AMI, instance type, instance profile, security group, and bootstrap/user-data script. Create an Auto Scaling group across private application subnets in both AZs. Set minimum, desired, and maximum capacity (for example 2/2/4 for a lab, subject to budget). Configure health checks and a scaling policy.

Step 8 — Create the Application Load Balancer

Create an internet-facing ALB in public subnets across two AZs. Create a target group using the app port and /health path. Add an HTTP listener for testing; for real use, configure HTTPS with an ACM certificate and redirect HTTP to HTTPS. Attach the target group to the Auto Scaling group.

Step 9 — Configure Route 53

Register or use a domain and create a hosted zone. Create an A/AAAA Alias record pointing to the ALB. DNS changes may take time to propagate. If you do not own a domain, test using the ALB DNS name.

Step 10 — Configure S3

Create a bucket with Block Public Access enabled and encryption enabled. Add a bucket policy only if required. Prefer CloudFront or controlled signed URLs for public delivery rather than making the entire bucket public. Test the application's authorized object access.

Step 11 — Configure CloudWatch

Review EC2, ALB, Auto Scaling, and RDS metrics. Publish application logs using the CloudWatch agent if needed. Create alarms for unhealthy ALB targets, elevated 5xx responses, high CPU, low free database storage, and relevant database health metrics. Configure an SNS notification target if desired.

Step 12 — Validate high availability

Verify that targets in both AZs are healthy. Stop or terminate one test EC2 instance and confirm Auto Scaling replaces it and the application remains available. For database failover testing, follow AWS RDS guidance and perform it only in a controlled test environment. Confirm alarms and logs work.

5. Validation checklist

- ☐ The ALB is deployed across at least two public subnets in different AZs.
- ☐ The target group reports healthy targets in multiple AZs.
- ☐ EC2 instances are in private application subnets and have no public IPs (recommended design).
- ☐ The RDS database is not publicly accessible and accepts port 3306 only from the application security group.
- ☐ The Auto Scaling group replaces a terminated test instance.
- ☐ The application can read/write the database as intended.
- ☐ S3 access is restricted and encryption is enabled.
- ☐ Route 53 resolves the domain to the ALB.
- ☐ CloudWatch logs, metrics, and alarms are visible.
- ☐ HTTPS is configured for any public-facing production deployment.

6. Cost and security notes

AWS resources may incur charges even when traffic is low. ALB, NAT Gateway, RDS Multi-AZ, data transfer, CloudWatch ingestion, and Route 53 hosted zones/domains can all contribute to cost. Set an AWS Budget, use the pricing calculator, choose small lab resources, and delete resources when finished. Deleting a database can permanently remove data; take a snapshot first if you need to keep it.

Security baseline: least-privilege IAM; private database; restrictive security groups; encrypted storage; TLS for public traffic; secrets stored securely; backups enabled; no credentials committed to GitHub; and logging/monitoring enabled.

7. Suggested evidence for your portfolio

- Architecture diagram with VPC, two AZs, subnets, ALB, EC2 Auto Scaling, RDS, and S3.
- VPC subnet and route-table screenshots (hide account IDs if desired).
- ALB target group showing healthy targets in two AZs.

- Auto Scaling group showing desired/min/max capacity.
- RDS configuration showing Multi-AZ and Public access disabled.
- CloudWatch dashboard or alarm screenshot.
- Browser screenshot of the working application using the Route 53 domain.

8. LinkedIn post template

□ AWS Cloud Project: Highly Available 3-Tier Web Application

I built a highly available web application architecture on AWS using VPC, EC2, Application Load Balancer, Auto Scaling, Amazon RDS for MySQL, S3, IAM, CloudWatch, and Route 53.

Key implementation highlights:

- Designed a VPC across multiple Availability Zones
- Routed traffic through an Application Load Balancer
- Configured EC2 Auto Scaling for instance replacement and capacity management
- Isolated the MySQL database in private subnets and configured Multi-AZ
- Applied IAM least privilege and security-group boundaries
- Added CloudWatch monitoring and Route 53 DNS

This project helped me practice AWS networking, high availability, security, monitoring, and cloud architecture.

Next steps: add HTTPS, automate deployment with Infrastructure as Code, and improve the monitoring dashboard.

#AWS #CloudComputing #AWSProjects #CloudEngineering #DevOps #LearningInPublic

Before posting: replace any claim that you have not implemented or tested. Do not share credentials, account IDs, IP addresses, database endpoints, or other sensitive details.

9. Official AWS documentation

- **Amazon VPC:** <https://docs.aws.amazon.com/vpc/>
- **Amazon EC2 Auto Scaling:** <https://docs.aws.amazon.com/autoscaling/ec2/>
- **Elastic Load Balancing:** <https://docs.aws.amazon.com/elasticloadbalancing/>
- **Amazon RDS for MySQL:**
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_MySQL.html
- **Amazon S3:** <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>
- **AWS IAM:** <https://docs.aws.amazon.com/IAM/latest/UserGuide/>
- **Amazon CloudWatch:** <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/>
- **Amazon Route 53:** <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/>
- **AWS Pricing Calculator:** <https://calculator.aws/>